

Scheme of Valuation/Answer Key			
(Scheme of evaluation (marks in brackets) and answers of problems/key)			
APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY			
SECOND SEMESTER B.TECH DEGREE EXAMINATION(2019 SCHEME), JUNE 2022			
Course Code: EST 102			
Course Name: PROGRAMMING IN C (Common to all programs)			
Max. Marks: 100			Duration: 3 Hours
PART A			
		Answer all Questions. Each question carries 3 Marks	Marks
1		What are the functions of ALU and CU? CPU is the brain of the computer system. All major calculation and comparisons are made inside the CPU and it is also responsible for activation and controlling the operation of other unit. This unit consists of two major components that are arithmetic logic unit (ALU) and control unit (CU). • Arithmetic Logic Unit (ALU): Here arithmetic logic unit performs all arithmetic operations such as addition, subtraction, multiplication and division. It also uses logic operation for comparison. • Control Unit (CU): The control unit of a CPU controls the entire operation of the computer. It also controls all devices such as memory, input/output devices connected to the CPU. The main work of the CU is to tell the most efficient method to work. It guides all the related operations to the individual parts of the processor. Some basic operation performed by it includes taking guidelines, controlling sequential execution, managing information stream in the correct course, directing and observing the time needed amid the preparing and	(3)

	taking care of different functions, etc. It controls the operations of the processor. It informs the ALU, I/O devices, and the RAM how to respond to the command that has been sent to the processor. A control unit directs the system and executes instructions entered by the user.	
2	Draw a flowchart to find the sum of first N numbers. Sum of first n numbers <pre> graph TD START([START]) --> ReadN[/Read n/] ReadN --> Init[sum = 0 i = 1] Init --> Decision{i <= n} Decision -- True --> LoopBody[sum = sum + i i = i + 1] LoopBody --> Decision Decision -- False --> PrintSum[/print sum/] PrintSum --> STOP([STOP]) </pre>	(3)
3	Differentiate between while and do-while loops using an example	(3)

WHILE	DO-WHILE
Condition is checked first then statement(s) is executed.	Statement(s) is executed, thereafter condition is checked.
It might occur statement(s) is executed zero times, If condition is false.	At least once the statement(s) is executed.
No semicolon at the end of while. while(condition)	Semicolon at the end of while. while(condition);
If there is a single statement, brackets are not required.	Brackets are always required.
while loop is entry controlled loop.	do-while loop is exit controlled loop.

<pre>while(condition) { statement(s); }</pre>	<pre>do { statement(s); } while(condition);</pre>
<pre>#include <stdio.h> void main() { int i=-1; while(i>0) { printf("Test"); } printf("End"); } Output End</pre>	<pre>#include <stdio.h> void main() { int i=-1; do { printf("Test"); } while(i>0); printf("End"); } Output Test End</pre>

4		Why is the use of goto statement discouraged in C programs? C is a highly structured language. goto is a unconditional control jump. This statement passes control anywhere in the program i.e. control is transferred to another part of the program without testing any condition. The goto statement is discouraged in C, because it alters the sequential flow of logic that is the characteristic of C language. Also it makes difficult to trace program flow control. The goto statement allows us to transfer control of the program to the specified label. Syntax: <pre>goto label; label: statement;</pre>	(3)
5		Write a C program to compare any two strings using string handling functions. <pre>include <stdio.h> #include <string.h> #include <stdlib.h> #define str_size 100 //Declare the maximum size of the string void main() {</pre>	(3)

```

char str1[str_size], str2[str_size];
int flg=0;
printf("Input the 1st string : ");
scanf("%s",str1);
printf("Input the 2nd string : ");
scanf("%s",str2);
int i=0;
while(str1[i] == str2[i])
{
    if(str1[i] == '\0' || str2[i] == '\0')
        break;
    i++;
}
if(str1[i-1] == '\0' && str2[i-1] == '\0')
    flg=0;
else if(str1[i] > str2[i])
    flg=1;
else if(str1[i] < str2[i])
    flg=-1;

if(flг == 0)
{
    printf("\nThe length of both strings are equal and \n also both strings are equal.\n\n");
}
else if(flг == -1)
{
    printf("\nThe length of the first string is smaller than second.\n\n");
}
else
{
    printf("\nThe length of the first string is greater than second.\n\n");
}
}

```

6	Write a C program to find the largest element in an array. Correct Program- 3 marks <pre> #include <stdio.h> int main() { int array[100], size, c, location = 0; printf("Enter the number of elements in array\n"); scanf("%d", &size); printf("Enter %d integers\n", size); for (c = 0; c < size; c++) scanf("%d", &array[c]); for (c = 1; c < size; c++) if (array[c] > array[location]) location = c; printf("Maximum element is present at location %d and its value is %d.\n", location+1, array[location]); return 0; } </pre>	(3)
7	Name the different types of parameter passing. Illustrate each of them with an example. Call by value & example- 1.5 marks Calling a function by value will cause the program to copy the contents of an object passed into a function. To implement this in C, a function declaration has the following form: [return type] functionName([type][parameter name],...). <pre> #include <stdio.h> void swap(int x, int y){ int temp = x; x = y; y = temp; } </pre>	(3)

```
int main(){
    int x = 10;
    int y = 11;
    printf("Values before swap: x = %d, y = %d\n", x,y);
    swap(x,y);
    printf("Values after swap: x = %d, y = %d", x,y);
}
```

Call by reference & example – 1.5 marks

Calling a function by reference will give function parameter the address of original parameter due to which they will point to same memory location and any changes made in function parameter will also reflect in original parameters. To implement this in C, a function declaration has the following form: [return type] functionName([type]* [parameter name],...).

```
include <stdio.h>
```

```
void swap(int *x, int *y){
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
int main(){
    int x = 10;
    int y = 11;
    printf("Values before swap: x = %d, y = %d\n", x,y);
```

	<pre> swap(&x,&y); printf("Values after swap: x = %d, y = %d", x,y); } </pre>	
8	What are the advantages of modular programming? Any two advantages 1.5 marks each Reusable Code - Divide a large software into multiple independent modules and make each class/method do just one specific task. Modularization makes code easy to understand and more maintainable. It allows easy reuse of methods or functions in a program and reduces the need to write repetitively. Ease of Maintenance - Having a program broken into smaller sub-programs allows for easier management. The separate modules are easier to test, implement, or design. These individual modules can then be used to develop the whole program. Easier to Debug - Debugging is one of the major parts of programming, and also one of the most irritating parts of development, When debugging large programs, how and when any bugs occur can become a mystery. This can take much of your valuable time as he searches through lines and lines of code to find out where an error occurred, however, then each discrete task has its discrete section of code. So, if there is a problem in a particular function, the programmer knows where to look and can manage a smaller portion of the code. Collaborate - Another advantage of this modular approach is that it allows for collaborative programming. Instead of giving a large job to a single programmer, you can split it up into a large team of programmers. Each programmer can be given a specific task to complete as part of the overall software application. Then, in the end, all of the various work from the programmers is compiled to create the program. This helps speed	(3)

		up the work.	
9		Distinguish between text mode and binary mode operation of a file. For Binary Mode The contents of the file are not structured in lines in the READ DATASET or TRANSFER operations. Instead, they are input or output as a stream. You do not have to specify the IN BINARY MODE addition explicitly. For Text Mode If you use this addition, the contents of the file are structured in lines. Each time you use the READ DATASET or TRANSFER statement, the system reads or writes a single line. If the data object to which you are transferring the data is too big, it is padded with spaces. If it is too small, the data record is truncated.	(3)
10		What do you mean by a pointer variable? How is it initialised? Pointer-1.5 marks It points the address of another variable. So the definition of pointer is that "the special type of variable which holds the address of another variable". Initialisation-1.5 marks	(3)

		Initialisation of pointer Syntax of pointer is data _type *pointer_ variable _name; In this syntax data type=int, char,float or double. *=symbol for pointer (asterisk).	
PART B			
<i>Answer any one Question from each module. Each question carries 14 Marks</i>			

11	a)	Explain linear search with an example. Draw a flowchart and write pseudo code to perform linear search on an array of numbers Explanation with example-4 marks Flowchart-4 marks Algorithm-6 marks Linear Search Searching is the process of finding some particular element in the list. If the element is present in the list, then the process is called successful, and the process returns the location of that element; otherwise, the search is called unsuccessful. Two popular search methods are Linear Search and Binary Search. Linear search is also called as sequential search algorithm. It is the simplest searching algorithm. In Linear search, we simply traverse the list completely and match each element of the list with the item whose location is to be found. If the match is found, then the location of the item is returned; otherwise, the algorithm returns NULL. It is widely used to search an element from the unordered list, i.e., the list in which items are not sorted. The steps used in the implementation of Linear Search are listed as follows - o First, we have to traverse the array elements using a for loop. - o In each iteration of for loop, compare the search element with the current array element, and - o If the element matches, then return the index of the corresponding array element. - o If the element does not match, then move to the next element. - o If there is no match or the search element is not present in the given array, return -1. Working of Linear search To understand the working of linear search algorithm, let's take an unsorted array. It will be easy to understand the working of linear search with an example.	(14)
----	----	--	------

Let the elements of array are -

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

Let the element to be searched is **K = 41**

Now, start from the first element and compare **K** with each element of the array.

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

↑
K ≠ 70

The value of **K**, i.e., **41**, is not matched with the first element of the array. So, move to the next element. And follow the same process until the respective element is found.

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

↑
K ≠ 40

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

↑
K ≠ 30

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

↑
K ≠ 11

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

↑
K ≠ 57

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

↑
K = 41

		Now, the element to be searched is found. So algorithm will return the index of the element matched. Algorithm Linear Search (Array A, Value x) Step 1: Set i to 1 Step 2: if i > n then go to step 7 Step 3: if A[i] = x then go to step 6 Step 4: Set i to i + 1 Step 5: Go to Step 2 Step 6: Print Element x Found at index i and go to step 8 Step 7: Print element not found Step 8: Exit Pseudocode <pre> procedure linear_search (list, value) for each item in the list if match item == value return the item's location end if end for end procedure </pre>	
		OR	
12	a)	a) List five important registers in CPU. Also state the purpose of each register. (6) Registers-1 Mark five important registers in CPU along with functions (5 marks)	(6)

- Accumulator:

This is the most frequently used register used to store data taken from memory. It is in different numbers in different microprocessors.

- Memory Address Registers (MAR):

It holds the address of the location to be accessed from memory. MAR and MDR (Memory Data Register) together facilitate the communication of the CPU and the main memory.

- Memory Data Registers (MDR):

It contains data to be written into or to be read out from the addressed location.

- General Purpose Registers:

These are numbered as R0, R1, R2....Rn-1, and used to store temporary data during any ongoing operation. Its content can be accessed by assembly programming. Modern CPU architectures tends to use more GPR so that register-to-register addressing can be used more, which is comparatively faster than other addressing modes.

- Program Counter (PC):

Program Counter (PC) is used to keep the track of execution of the program. It contains the memory address of the next instruction to be fetched. PC points to the address of the next instruction to be fetched from the main memory when the previous instruction has been

		successfully completed. Program Counter (PC) also functions to count the number of instructions. The incrementation of PC depends on the type of architecture being used. If we are using 32-bit architecture, the PC gets incremented by 4 every time to fetch the next instruction. ● Instruction Register (IR): The IR holds the instruction which is just about to be executed. The instruction from PC is fetched and stored in IR. As soon as the instruction is placed in IR, the CPU starts executing the instruction and the PC points to the next instruction to be executed.	
	b)	b) Write an algorithm to find sum of digits of a number. (8) Correct Algorithm- 7 marks 1. Input a Number 1. Initialize Sum to zero 2. While Number is not zero 3. Get Remainder by Number Mod 10 4. Add Remainder to Sum 5. Divide Number by 10 6. Print sum	(8)

13	a)	Any 4 data types with explanation - 4 marks. Memory requirements- 3 marks. int or signed int-2bytes Integers are whole numbers that can have both zero, positive and negative values but no decimal values. Float -4 bytes Float and double are used to hold real numbers. The floating-point data type allows the user to type decimal values. For example, average marks can be 97.665. if we use int data type, it will strip off the decimal part and print only 97. To print the exact value, we need 'float' data type. Char-1 byte	(7)
----	----	---	-----

	char stores a single character. Char consists of a single byte. Double-8 bytes Double is 8 bytes, which means you can have more precision than float. This is useful in scientific programs that require precision. Float is just a single-precision data type; double is the double-precision data type.	
b)	Write down a C program to check if a number is present in a given list of numbers. If present give location of the number otherwise insert the number in the list at the end. Ans: <pre>#include<stdio.h> int main() { int a[20],target,cnt,n; cnt=0; printf("Enter the No. of Elements :"); scanf("%d",&n); printf("Enter the target :"); scanf("%d",&target); printf("Enter the Elements :"); while(cnt<n) { scanf("%d",&a[cnt]); if(target==a[cnt]) { printf("Found at position : %d",cnt+1);</pre>	(7)

		<pre> return 0; } cnt++; } printf("Not Found"); a[cnt]=target; printf("The new Elements of array :"); for(cnt=0;cnt<n+1;cnt+ +) { printf("%d\t",a[cnt]); } return 0; } </pre>	
		OR	
14	a)	formatted I/O functions with example-3.5 marks Formatted I/O Functions Formatted I/O functions are used to take various inputs from the user and display multiple outputs to the user. These types of I/O functions can help to display the output to the user in different formats using the format specifiers. These I/O supports all data types like int, float, char, and many more. printf(): printf() function is used in a C program to display any value like float, integer, character, string, etc on the console screen. It is a pre-defined function that is already declared in the stdio.h(header file). Syntax 1: To display any variable value.	(7)

	<pre>printf("Format Specifier", var1, var2,, varn);</pre>	
--	---	--

	<pre>scanf():</pre>	
--	---------------------	--

	scanf() function is used in the C program for reading or taking any value from the keyboard by the user, these values can be of any data type like integer, float, character, string, and many more. This function is declared in stdio.h(header file), that's why it is also a pre-defined function. In scanf() function we use &(address-of operator) which is used to store the variable value on the memory location of that variable.	
--	---	--

	Syntax:	
--	----------------	--

	<pre>scanf("Format Specifier", &var1, &var2,, &varn);</pre>	
--	---	--

	Unformatted I/O functions with example-3.5 marks	
--	---	--

	Unformatted Input/Output functions	
--	---	--

	Unformatted I/O functions are used only for character data type or character array/string and cannot be used for any other datatype. These functions are used to read single input from the user at the console and it allows to display the value at the console.	
--	---	--

- | | | |
|--|--|--|
| | <ol style="list-style-type: none">1. getch()2. getche()3. getchar()4. putchar()5. gets()6. puts() | |
|--|--|--|

7. putchar()

getch():

getch() function reads a single character from the keyboard by the user but doesn't display that character on the console screen and immediately returned without pressing enter key. This function is declared in conio.h(header file). getch() is also used for hold the screen.

Syntax:

```
getch();
```

or

```
variable-name = getch();
```

getchar():

The getchar() function is used to read only a first single character from the keyboard whether multiple characters is typed by the user and this function reads one character at one time until and unless the enter key is pressed. This function is declared in stdio.h(header file)

Syntax:

```
Variable-name = getchar();
```

putchar():

The putchar() function is used to display a single character at a time by passing that character directly to it or by passing a variable that has already stored a character. This function is declared in stdio.h(header file)

Syntax:

	putchar(variable_name); gets(): gets() function reads a group of characters or strings from the keyboard by the user and these characters get stored in a character array. This function allows us to write space-separated texts or strings. This function is declared in stdio.h(header file). Syntax: char str[length of string in number]; //Declare a char type variable of any length gets(str); puts(): In C programming puts() function is used to display a group of characters or strings which is already stored in a character array. This function is declared in stdio.h(header file). Syntax: puts(identifier_name);	
	b) Write a C program to read a character from the user and check whether it is a vowel or consonant.	(7)

		<pre> #include <stdio.h> int main() { char ch; printf("Enter a character\n"); scanf("%c", &ch); // Checking both lower and upper case, is the OR operator if (ch == 'a' ch == 'A' ch == 'e' ch == 'E' ch == 'i' ch == 'I' ch == 'o' ch == 'O' ch == 'u' ch == 'U') printf("%c is a vowel.\n", ch); else printf("%c isn't a vowel.\n", ch); return 0; } </pre>	
15	a)	Write a C program to find the transpose of a matrix. <pre> #include <stdio.h> int main() </pre>	(7)

		<pre> { int m, n, c, d, matrix[10][10], transpose[10][10]; printf("Enter the number of rows and columns of a matrix\n"); scanf("%d%d", &m, &n); printf("Enter elements of the matrix\n"); for (c = 0; c < m; c++) for (d = 0; d < n; d++) scanf("%d", &matrix[c][d]); for (c = 0; c < m; c++) for (d = 0; d < n; d++) transpose[d][c] = matrix[c][d]; printf("Transpose of the matrix:\n"); for (c = 0; c < n; c++) { for (d = 0; d < m; d++) printf("%d\t", transpose[c][d]); printf("\n"); } return 0; } </pre>	
	b)	4 string handling functions along with Syntax -7 marks	(7)

- | | | |
|--|--|--|
| | <ul style="list-style-type: none"> • <u>String Copy:</u> <code>strcpy(s1,s2);</code> <ul style="list-style-type: none"> • Copies s1 to s2 • To be used instead of assignment operator • <u>String Concatenation:</u> <code>strcat(s1,s2);</code> <ul style="list-style-type: none"> • Combines s2 to s1 • Stores resultant string in s1 • s2 remains unchanged. • Size of s1 must be large enough to accommodate the final result. • <u>String Comparison:</u> <code>strcmp(s1,s2);</code> <ul style="list-style-type: none"> • Compares s1 and s2, and returns their difference(integer) • To be used instead of assignment operator • Return value <ul style="list-style-type: none"> 0, if s1 == s2 1 or more, if s1 > s2 -1 or less, if s2 > s1 • <u>String Length:</u> <code>strlen(s1);</code> <ul style="list-style-type: none"> • Finds the length of given string | |
|--|--|--|

		OR	
16	a)	Write a C program to reverse a string without using string handling functions. #include <stdio.h>	(7)

	<pre> #include <string.h> int main() { char Str[100], RevStr[100]; int i, j, len; printf("\n Please Enter any String : "); gets(Str); j = 0; len = strlen(Str); for (i = len - 1; i >= 0; i--) { RevStr[j++] = Str[i]; } RevStr[i] = '\0'; printf("\n String after Reversing = %s", RevStr); return 0; } </pre>	
b)	Write a C program to sort an array of numbers using bubble sort. <pre> #include<stdio.h> int main() { int i; int a[5],search=0; for(i=0;i<5;i++) scanf("%d",&a[i]); for(i=0;i<5;i++) </pre>	(7)

		<pre> { for(int j=0;j<5-i-1;j++) { if(a[j]>a[j+1]) { int t=a[j]; a[j]=a[j+1]; a[j+1]=t; } } } for(i=0;i<5;i++) { printf("%d \t ",a[i]); } printf("\n"); return 0; } </pre>	
17	a)	Write a C program to : (i) Create a structure containing the fields: Name, Price, Quantity, Total Amount. (ii) Use separate functions to read and print the data. Struct Product <pre> { char Name[15]; float Price; int Quantity; int TotalAmount; </pre>	(7)

		<pre> } void read(Struct S) { scanf("%s",S.Name); scanf("%f", &S.Price); scanf("%d",&S.Quantity); scanf("%d",&S.TotalAmount); } void display() { printf("%s",S.Name); printf("%f", S.Price); printf("%d",S.Quantity); printf("%d",S.TotalAmount); } </pre>	
	b)	What are different storage classes in C? Give examples for each. Storage class definition -1 mark. Any 3 storage classes (Auto, extern, static, register)- 1 mark each.	(7)

Example – 1 Mark each

Storage class refers to the performance of a variable, and its scope within the program, i.e., the portion of the program over which the variable is recognized.

- There are four different storage-class specifications in C:

- Automatic variables
- External variables
- Static variables
- Register variables

Automatic Variables

- Automatic variables are declared inside a function in which they are to be utilized.
- They are created when the function is called and destroyed automatically when the function is exited, hence the name automatic.
- Automatic variables are therefore private (or local) to the function in which they are declared.
- The automatic variables are initialized to garbage by default.
- Because of this property, automatic variables are also referred to as local or internal variables.
- A variable declared inside a function without storage class specification is, by default, an automatic variable.
- General form is: < storage class specifier > < data type > < variable name > ;
 - Ex: auto char name ; auto int number ;

External variables

- Variables that are both alive and active throughout the entire program are known as external variables. • Also known as global variables.
- External variables are declared outside a function.
- The external storage class is used to tell the compiler that the variable defined as extern is declared with an external linkage elsewhere in the program.
- The variables declared as extern are not allocated any memory. It is only declaration and intended to specify that the variable is declared elsewhere in the program.

		• The default initial value of external integral type is 0 otherwise null. • We can only initialize the extern variable globally, i.e., we can not initialize the external variable within any block or method. • An external variable can be declared many times but can be initialized at only once. • If a variable is declared as external then the compiler searches for that variable to be initialized somewhere in the program which may be extern or static. If it is not, then the compiler will show an error. Eg: extern int a; Static variables • The value of static variables persists until the end of the program. • A variable can be declared static using the keyword static. <pre>static int x; static float y;</pre> • A static variable may be either an internal type or an external type depending on the place of declaration. Internal static variables are those which are declared inside a function. The scopes of these variables extend up to the end of the function in which they are defined. • Therefore internal static variables are similar to auto variables, except that they remain in existence (alive) throughout the remainder of the program. Therefore, internal static variables can be used to retain values between function calls. • An external static variable is declared outside of all functions and is available to all the functions in that program.	
		OR	
18	a)	What are the main differences between structures and unions? Which is preferred in what situation? Give examples. Differences-3 marks Situation -2 marks Example- 2 marks	(7)

	STRUCTURE	UNION
Keyword	The keyword struct is used to define a structure	The keyword union is used to define a union.
Size	When a variable is associated with a structure, the compiler allocates the memory for each member. The size of structure is greater than or equal to the sum of sizes of its members.	when a variable is associated with a union, the compiler allocates the memory by considering the size of the largest memory. So, size of union is equal to the size of largest member.
Memory	Each member within a structure is assigned unique storage area of location.	Memory allocated is shared by individual members of union.
Value Altering	Altering the value of a member will not affect other members of the structure.	Altering the value of any of the member will alter other member values.
Accessing members	Individual member can be accessed at a time.	Only one member can be accessed at a time.
Initialization of Members	Several members of a structure can initialize at once.	Only the first member of a union can be initialized.

structure Eg:

```
struct [structure name]
{
    member definition;
    member definition;
    ...
    member definition;
}structure variable declaration;
```

Union Example:

```
union [union name]
{
    member definition;
    member definition;
    ...
    member definition;
```

		<code>}union variable declaration;</code>	
	b)	What is recursion? Write a C program to display Fibonacci series using recursive function. Recursion- 1 mark Correct Program- 6 marks Recursion is the mechanism by which a function calls itself • The use of recursion improves the readability of the program, but the overhead of processor is increased. When we write recursive functions, we must have an if statement somewhere to force the function to return without the recursive call being executed. Otherwise, the function will never return. • Recursion cannot be applied to all the problem, but it is more useful for the tasks that can be defined in terms of similar subtasks. For Example, recursion may be applied to sorting, searching, and traversal problems. • Generally, iterative solutions are more efficient than recursion since function call is always overhead. Any problem that can be solved recursively, can also be solved iteratively. However, some problems are best suited to be solved by the recursion, for example, tower of Hanoi, Fibonacci series, factorial finding, etc. <pre> #include<stdio.h> int f(int); int main() { int n, m= 0, i; printf("Enter Total terms:n"); </pre>	(7)

		<pre> scanf("%d", &n); printf("Fibonacci series terms are:n"); for(i = 1; i <= n; i++) { printf("%dn", fibonacci(m)); m++; } return 0; } int fibonacci(int n) { if(n == 0 n == 1) return n; else return(fibonacci(n-1) + fibonacci(n-2)); } </pre>	
19	a)	Write a C program to replace vowels in a text file with character 'x'. Correct Program- 7 marks <pre> #include<stdio.h> #include<conio.h> int main() { char str[50], ch='x', i; printf("Enter any string: "); gets(str); for(i=0; str[i]!='\0'; i++) { if(str[i]=='a' str[i]=='e' str[i]=='i' str[i]=='o' </pre>	(7)

	<pre> str[i]=='u' str[i]=='A' str[i]=='E' str[i]=='I' str[i]=='O' str[i]=='U') { str[i] = ch; } } printf("\nNew String (after replacing vowel with %c) = %s", ch, str); getch(); return 0; } </pre>	
b)	Explain how pointers can be passed to functions in C. Explanation with a sample program- 7 marks Passing Pointer to a Function When we pass a pointer as an argument instead of a variable then the address of the variable is passed instead of the value. So any change made by the function using the pointer is permanently made at the address of passed variable. This technique is known as call by reference in C. <pre> #include <stdio.h> void salaryhike(int *var, int b) { *var = *var+b; } int main() { int salary=0, bonus=0; printf("Enter the employee current salary:"); scanf("%d", &salary); printf("Enter bonus:"); </pre>	(7)

		<pre>scanf("%d", &bonus); salaryhike(&salary, bonus); printf("Final salary: %d", salary); return 0; }</pre>	
		OR	
20	a)	Any 5 file handling functions with syntax-7 marks <pre>fopen() fgets() fprintf() fclose() fseek()</pre>	(7)
	b)	Write a C program to print the elements of an array in reverse order using pointers. <pre>#include <stdio.h> void main() { int n, i, arr1[15]; int *pt; printf("\n\n Pointer : Print the elements of an array in reverse order :\n");</pre>	(7)

```

printf(" Input the number of elements to store in the array (max 15) : ");
scanf("%d",&n);
pt = &arr1[0];
printf(" Input %d number of elements in the array : \n",n);
for(i=0;i<n;i++)
{
    printf(" element - %d : ",i+1);
    scanf("%d",pt);
    pt++;
}

pt = &arr1[n - 1];
printf("\n The elements of array in reverse order are :");
for (i = n; i > 0; i--)
{
    printf("\n element - %d : %d ", i, *pt);
    pt--;
}
printf("\n\n");
}

```